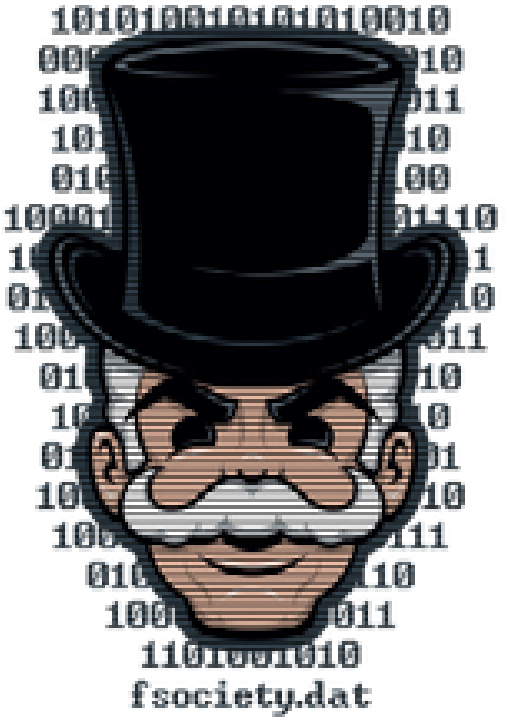
PENETRATION TEST REPORT

MR. ROBOT



Target: 10.67.152.60 (THM Machine-Linux Environment)

Type: Black Box Engagement

Classification: CONFIDENTIAL

Prepared by: Kiron García (Lead Security Researcher)

Contact: kiron.garcia.trabajo@gmail.com

Timeline: 24/03/2026 — [29/03/2026]

Version: 1.0

TABLE OF CONTENTS

EXECUTIVE SUMMARY

This report documents the findings of a black-box penetration test conducted against the Mr. Robot target (10.67.152.60) within the TryHackMe controlled lab environment. The engagement was authorized via platform subscription and executed under the Penetration Testing Execution Standard (PTES) and OWASP Testing Guide v4, across five structured phases: Pre-Engagement & Scoping, Intelligence Gathering, Vulnerability Analysis, Exploitation, and Post-Exploitation & Privilege Escalation.

The assessment identified nine findings: four Critical, four High, and one Medium. Full system compromise was achieved through an unbroken chain of publicly documented misconfigurations — no zero-day exploitation or custom tooling was required. The attack chain originated at an unauthenticated robots.txt disclosure (F-01), progressed through a decade-old WordPress installation (F-02), leveraged an exposed Theme Editor to execute a reverse shell (F-03, F-04), and concluded with privilege escalation to root via a SUID legacy binary (F-05). Post-exploitation confirmed cascading credential exposure across all service layers: OS-level hashes, database credentials, and FTP plaintext passwords were simultaneously compromised (F-06, F-07, F-08). A persistence mechanism was successfully implanted and validated (F-09).

The primary risk driver is not the sophistication of individual vulnerabilities but the absence of foundational hardening controls: no account lockout policy, no file editor restriction, no SUID audit, and a CMS version abandoned by its vendor in 2015. Immediate remediation priority must be assigned to the full IMMEDIATE-tier finding set (F-02 through F-08). STRATEGIC-tier items (F-01, F-09) should follow within the next sprint cycle. Full remediation of the identified chain would reduce the exploitable attack surface to near-zero for this asset.

METHODOLOGY

The security assessment is conducted following the **Penetration Testing Execution Standard (PTES)** and the **OWASP Testing Guide (v4)** frameworks. The engagement is executed through a structured five-phase process to ensure comprehensive coverage of the attack surface:

1. **Pre-Engagement & Scoping:** This is where the client's rules are defined, along with the permitted scope for the audit.

2. **Intelligence Gathering & Reconnaissance:** Collecting data through passive (OSINT) and active methods to map the target infrastructure.
3. **Vulnerability Analysis:** Systematically identifying security weaknesses and misconfigurations in the exposed services.
4. **Exploitation:** Safely attempting to leverage identified vulnerabilities to gain initial unauthorized access.
5. **Post-Exploitation & Privilege Escalation:** Assessing the impact of the access gained, performing internal enumeration, and attempting to reach administrative (root) privileges.

Stealth & Evasion Approach: To simulate a real-world sophisticated threat and avoid triggering automated defense mechanisms (IDS/WAF), a **low-and-slow** approach is applied. This includes utilizing Nmap polite timing (T2), implementing delays between web requests, and prioritizing manual enumeration over aggressive automated scanning.

PHASE 1 — PRE-ENGAGEMENT & SCOPING

Engagement Details

- **Frameworks:** PTES (Penetration Testing Execution Standard) & OWASP TGv4.
- **Strategy:** Stealth-focused (T2/T3 Nmap, request delays) to bypass IDS/Rate-limiting.
- **Authorization:** Explicitly granted via platform subscription for 10.67.152.60.

Scope & Rules of Engagement

In-Scope	Out-of-Scope / Prohibited
10.67.152.60	DoS/DDoS attacks
All exposed TCP/UDP services	IPs outside target scope
Web Application & OS	Infrastructure/platform attacks

Auditor's Note: Due to the dynamic nature of the THM virtual lab, the target IP address may rotate during the assessment. All documented evidence (screenshots, logs, etc.) remains valid for the scoped asset "Mr. Robot", regardless of the specific IP shown in the terminal.

Project Objectives

1. **Surface Mapping:** Comprehensive passive/active reconnaissance.
2. **Risk Validation:** Identification of exploitable vulnerabilities (OWASP/NIST).
3. **Compromise:** Full system takeover (Root) via privilege escalation.
4. **Remediation:** Delivery of actionable technical recommendations.

PHASE 2 — INTELLIGENCE GATHERING

2.1 Passive Reconnaissance (OSINT)

A passive information gathering phase was conducted to map the attack surface without direct interaction, ensuring no detection by monitoring systems or WAF/IDS alerts.

External Surface Mapping

- **Shodan / Censys:** No public indexing detected (Confirmed Isolated Environment).
- **crt.sh (SSL):** No public certificates identified for this asset.
- **Whois:** N/A — Private IP space assigned to the lab environment.

Positive Security Observations (PSO) The following security controls were found to be correctly implemented:

- **PSO-01 — Banner Masking:** The `Server: Apache` header successfully suppresses specific versioning and OS distribution strings, mitigating immediate service fingerprinting.
- **PSO-02 — Clickjacking Protection:** The `X-Frame-Options: SAMEORIGIN` header is active, preventing the application from being embedded in malicious external frames.

Source Code Analysis Manual inspection of the application's front-end source code was performed to identify hidden metadata or hardcoded configurations.

- **Asset Identity:** Custom ASCII art was identified in the HTML comments, confirming the asset's alignment with the "Mr. Robot" project identity.
- **Information Leakage:** A sensitive JavaScript variable `USER_IP` was found hardcoded in the `<head> section, exposing a potential internal or proxy infrastructure IP (208.185.115.6`).
- ***Auditor's Note:*** *For identified artifacts (encoded USER_IP variable, custom ASCII metadata in HTML comments), it is recommended that the development process include a pre-deployment step to remove debug variables, inline comments, and encoded values from client-side code before it goes into production.*

[ID: F-01] MEDIUM: Sensitive Resource Disclosure via robots.txt

Finding Overview Severity: MEDIUM (5.3) — **Asset:** 10.65.140.35 (THM-Dynamic IP)
Vector: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`
Service: HTTP (Apache) — **CWE:** CWE-538

Risk Mapping & Compliance

- **ASVS:** V4.1.3 (Sensitive files in web root)
- **NIST CSF:** PR.IP-1 (Baseline configuration management)
- **Business Impact:** High. The exposure of internal wordlists and sensitive keys directly facilitates targeted brute-force attacks and reduces attacker reconnaissance time.

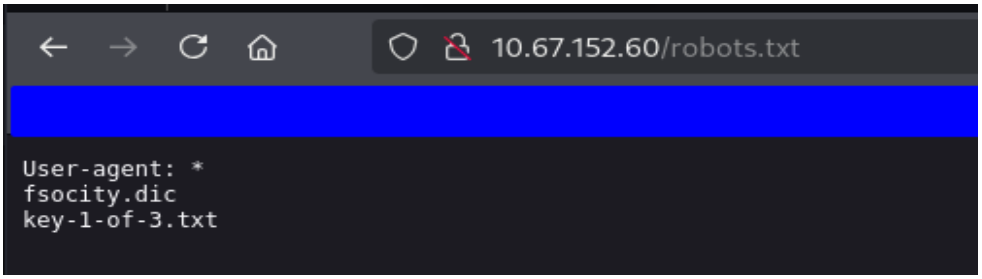
Attack Narrative

During manual inspection of the `robots.txt` file—a standard entry point for web reconnaissance—it was identified that the server explicitly indexes sensitive paths. The most critical exposure is `fsocity.dic`, a wordlist containing thousands of entries. An attacker can immediately weaponize this file to initiate credential-stuffing or brute-force attacks against any authentication interface within the environment.

Technical Evidence & Documentation

1. Resource Discovery & Data Exfiltration Unauthenticated access to sensitive resources advertised via the robots exclusion protocol.

- **Target URL:** `[http://10.65.140.35/robots.txt](http://10.65.140.35/robots.txt)`
- **Identified Resources:**
  - `/fsocity.dic` (Downloadable wordlist).
  - `/key-1-of-3.txt` (Internal sensitive token/key).




```
└─# wget http://10.66.175.15/fsocity.dic -O fsocity.dic
--2026-03-29 08:38:26--  http://10.66.175.15/fsocity.dic
Connecting to 10.66.175.15:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 7245381 (6.9M) [text/x-c]
Saving to: 'fsocity.dic'

fsocity.dic                 100%[=============================================>]

2026-03-29 08:38:36 (732 KB/s) - 'fsocity.dic' saved [7245381/7245381]
```

Remediation Strategy

- **[IMMEDIATE] Resource Isolation:** Move `fsocity.dic` and `key-1-of-3.txt` outside of the public web root (`/var/www/html`). Sensitive assets must never be accessible via a direct URL.
- **[STRATEGIC] Robots.txt Hardening:** Sanitize `robots.txt` to only include paths relevant for search engine crawlers. Never use this file to "hide" sensitive directories, as it serves as a public directory map for attackers.
- **[TECHNICAL NOTE] Fingerprinting Mitigation:** Although informational, it was noted that the `X-Mod-Pagespeed` version (1.9.32.3) is exposed. It is recommended to suppress this header to further harden the server against version-specific exploits.

Effort: Low | **Impact:** Medium (Surface Reduction)

2.2 Active Reconnaissance

An active scanning phase was executed to identify open ports, service versions, and hidden directories. To maintain a low profile, a "Polite" timing strategy was initially adopted, followed by specialized fingerprinting once a hardened environment was detected.

Service Enumeration & Infrastructure Mapping A full 65,535 TCP port scan was performed to ensure no "Security by Obscurity" services were active.

Endpoint Discovery:

- **Port 22/TCP (SSH):** Running `OpenSSH 8.2p1 (Ubuntu 4ubuntu0.1)`. This service represents a potential entry point for credential-based attacks (Brute Force/Leaked Keys).
- **Port 80/TCP (HTTP):** Managed by `Apache httpd`. Fingerprinting confirmed an active **WordPress 4.3.1** installation, identifying the primary web application attack surface.
- **Port 443/TCP (HTTPS/SSL):** `Apache httpd` detected with an **Expired SSL Certificate** (Sept 2025), indicating a lapse in infrastructure maintenance.

Positive Security Observations (PSO)

- **PSO-03 — Access Control & Hardening:** Standard user enumeration techniques via Author IDs (`/?author=1`) and REST API endpoints (`/wp-json/v2/users`) returned 404/403 errors, indicating active hardening against automated discovery.
- **PSO-04 — Deceptive Defense:** Critical files such as `readme.html` and `license.txt` were manually modified with custom strings to mislead automated scanners and discourage manual inspection.

[ID: F-02] HIGH: Outdated CMS & Infrastructure Information Leakage

Finding Overview Severity: HIGH (7.5) — **Asset:** 10.65.169.135 (THM-Dynamic IP)
Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Service: HTTP (Apache) — **CWE:** CWE-1104 (Use of Unmaintained Third-Party Components)

Risk Mapping & Compliance

- **ASVS:** V14.2 (Dependency Management)
- **NIST CSF:** ID.AM-2 (Software platforms and applications inventory)
- **Business Impact:** High. The use of a decade-old CMS version (2015) exposes the organization to hundreds of known, weaponized exploits (RCE, SQLi, Auth Bypass) that are publicly available.

Attack Narrative

The phase began with a **Full TCP Port Scan (1-65535)** via Nmap to identify the complete attack surface beyond standard ports. This identified an active Apache web server and an expired SSL certificate on port 443, signaling a lack of infrastructure maintenance.

A **"Silent-First" manual inspection** was then conducted on Port 80. Accessing `/readme.html` and `/license.txt` revealed a **Deceptive Security posture**, where the administrator manually modified these files with hostile strings to obfuscate the system's version.

To bypass this obfuscation, the auditor pivoted to **Specialized Fingerprinting (WPScan)** using a spoofed `User-Agent`. This "energetic" measure successfully unmasked the legacy environment: a **WordPress 4.3.1 (2015)** installation. The discovery of an active `xmlrpc.php` endpoint further expanded the attack surface, providing a confirmed vector for the next phase of exploitation.

Technical Evidence & Documentation

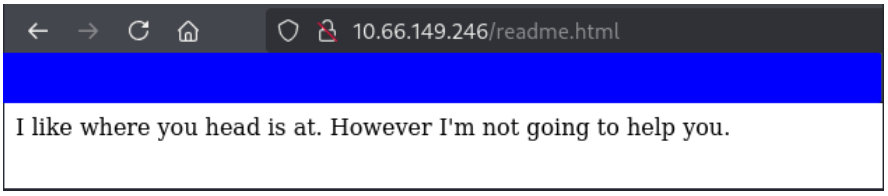
1. Service & Port Discovery (Nmap) Full TCP port discovery used to identify active services and version banners.

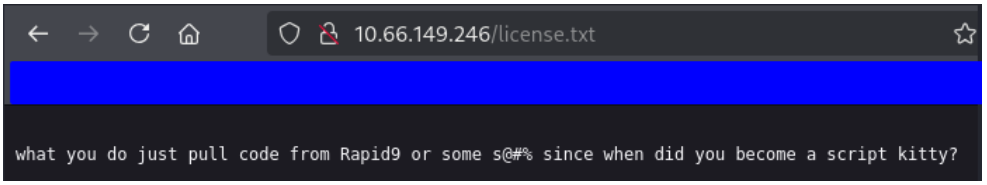
- **Command:** `sudo nmap -sS -Pn -n -T3 -p- --open -sV -sC 10.65.169.135`
- **Key Result: SSH (22), HTTP (80/Apache), HTTPS (443/Expired SSL).**

```
└─# sudo nmap -sS -Pn -n -T3 -p- --open -sV -sC -oN nmap_report.txt 10.65.169.135
[sudo] password for kiron:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-26 11:03 -05
Nmap scan report for 10.65.169.135
Host is up (0.12s latency).
Not shown: 65532 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT    STATE SERVICE  VERSION
22/tcp  open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 a9:61:83:35:9c:f3:b3:fc:30:71:ae:2d:51:d3:48:95 (RSA)
|   256 b4:bf:83:f7:df:ee:66:db:cc:02:b2:02:38:e6:b2:e4 (ECDSA)
|_  256 59:66:67:2c:38:5b:a5:b7:34:b4:b6:65:27:ed:ca:3d (ED25519)
80/tcp  open  http     Apache httpd
|_http-server-header: Apache
|_http-title: Site doesn't have a title (text/html).
443/tcp open  ssl/http Apache httpd
| ssl-cert: Subject: commonName=www.example.com
| Not valid before: 2015-09-16T10:45:03
|_Not valid after:  2025-09-13T10:45:03
|_http-server-header: Apache
```

2. Manual Obfuscation Analysis Manual inspection of root documentation files confirmed intentional hardening/deception.

- **Target Path:** `/readme.html` & `/license.txt`
- **Response: Custom strings used to hide versioning (e.g., *"I'm not going to help you"*).**





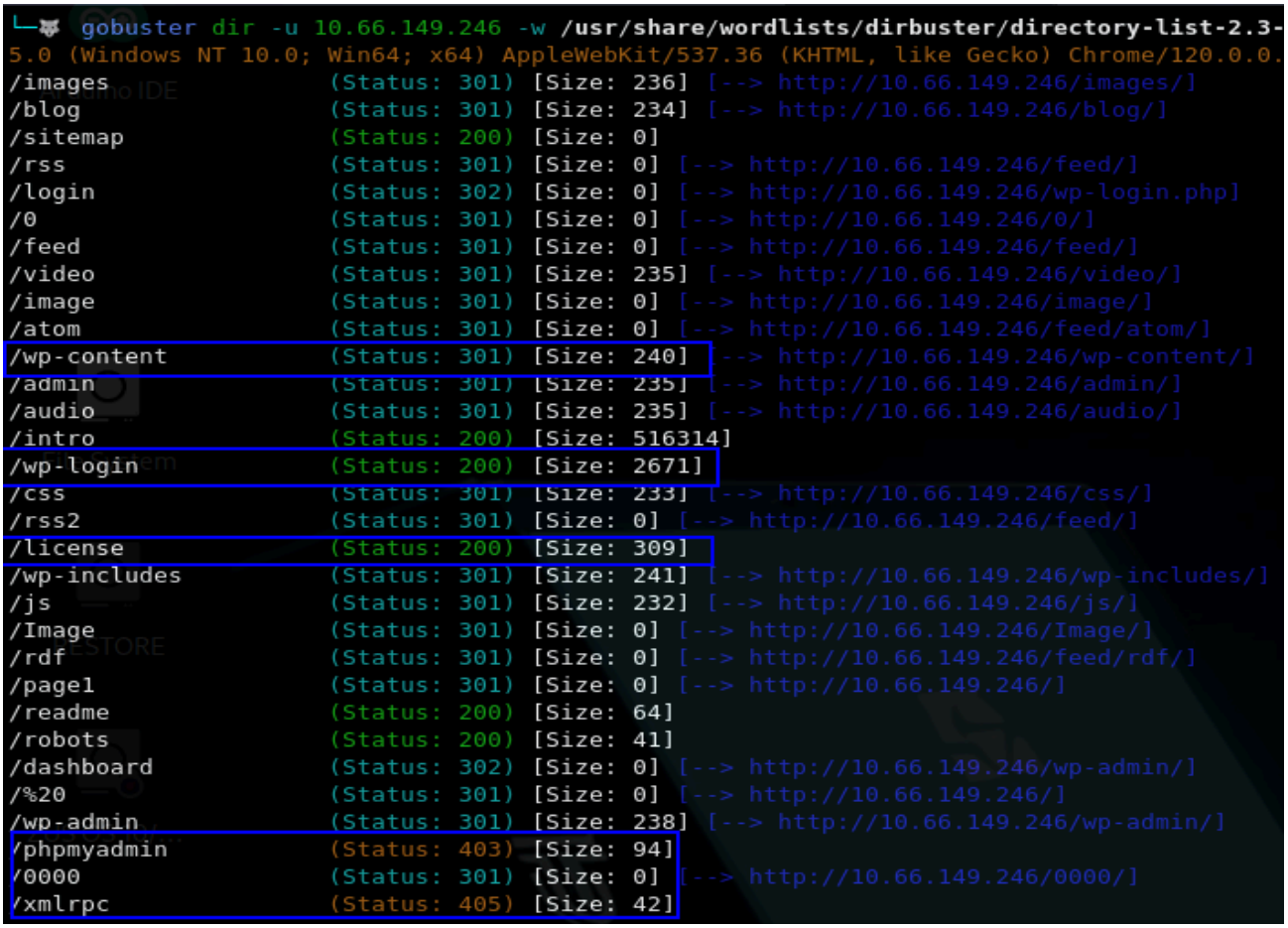
3. Specialized CMS Fingerprinting (WPScan) Automated identification of the WordPress core version through passive metadata analysis.

- **Command:** `wpscan --url [http://10.66.149.246/](http://10.66.149.246/) --enumerate u,vp --detection-mode mixed`
- **Identified Version:** WordPress 4.3.1 (Legacy).
- **Vulnerability Metadata: Confirmed via** `wp-emoji-release.min.js?ver=4.3.1`*WPScan Version & Metadata Hits*

```
[+] WordPress version 4.3.1 identified (Insecure, released on 2015-09-15).
| Found By: Emoji Settings (Passive Detection)
|  - http://10.66.158.245/b79fdcb.html, Match: 'wp-includes\/js\/wp-emoji-release.min.js?ver=4.3.1
| Confirmed By: Meta Generator (Passive Detection)
|  - http://10.66.158.245/b79fdcb.html, Match: 'WordPress 4.3.1'
```

4. Administrative Endpoint Mapping Directory discovery identified critical login and management interfaces.

- **Critical Paths:** `/wp-login.php` (200 OK), `/xmlrpc.php` (405 Method Not Allowed), `/phpmyadmin` (403 Forbidden).



Remediation Strategy

- **[IMMEDIATE] Emergency Update:** Upgrade the WordPress core to the latest stable version (6.x) immediately. Security patches for v4.3.1 have been discontinued for years.
- **[STRATEGIC] XML-RPC Disabling:** Disable `xmlrpc.php` via `.htaccess` or a security plugin if not required for external API integrations to mitigate brute-force risks.
- **[TECHNICAL NOTE] SSL Lifecycle Management:** The certificate on port 443 is expired (2025-09-13). It is recommended to implement an automated renewal process (e.g., Let's Encrypt) to maintain encrypted traffic integrity.

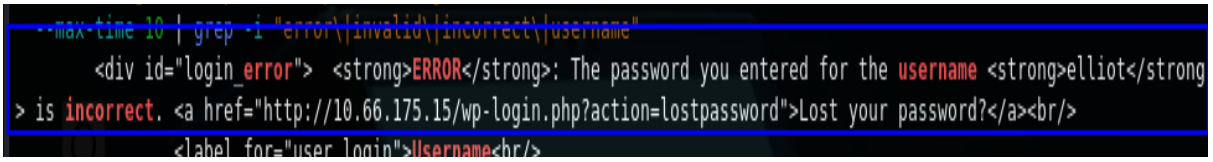
Effort: Medium | **Impact:** High (Total Attack Surface Reduction)

PHASE 3 — VULNERABILITY ANALYSIS

With the attack surface established in Phase 2, this phase focused on validating and exploiting identified vulnerabilities while preserving the low-noise approach adopted throughout the engagement.

Credential Discovery & User Validation

A single-request differential technique against `/wp-login.php` was used to silently confirm the username `elliot` — WordPress 4.3.1 returns distinct error messages for invalid usernames versus incorrect passwords, enabling user validation without automated scanners. The candidate was identified through contextual intelligence associated with the target's thematic identity. In a real-world engagement, this step would follow a structured OSINT phase covering social media and professional networks. Had no candidate been available, passive WPScan enumeration would have been the recommended fallback.



Positive Security Observations (PSO)

- **PSO-05 — Minimal Plugin Exposure:** Passive enumeration revealed no vulnerable plugins installed beyond the core CMS. The attack surface is contained to the WordPress core itself, reducing the overall exploitable footprint.

[ID: F-03] CRITICAL: Unauthenticated PHP Execution via Exposed Theme Editor

Finding Overview Severity: CRITICAL (9.9) | **Asset:** 10.66.131.120 (THM-Dynamic IP)
Vector: CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:C/C:H/I:H/A:H |
Service: HTTP (WordPress Admin) | **CWE:** CWE-94

Risk Mapping & Compliance

- **ASVS:** V12.3.1 (File execution from untrusted sources)
- **NIST CSF:** PR.AC-4 (Access permissions managed incorporating least privilege)
- **Business Impact:** Critical. Any authenticated administrator can execute arbitrary OS-level code through the browser with no additional tooling, enabling data exfiltration, persistent backdoor installation, and lateral movement.

Attack Narrative

Manual inspection of the WordPress administration panel revealed that the native Theme Editor component exposes direct PHP file editing capabilities to any authenticated administrator — with no additional authorization layer, no file-type restriction, and no execution sandbox. The file `404.php` was identified as an optimal injection target: trivially triggerable via a single HTTP request to any non-existent URL, and generating minimal log entries. This vulnerability, combined with the user enumeration weakness and the absence of account lockout controls, constitutes a complete exploitation chain pending valid credentials.

Technical Evidence & Documentation

1. Theme Editor Exposure Validation Manual navigation confirmed the Theme Editor is active and permits direct modification of server-side PHP files without secondary confirmation or privilege escalation.

- **Path:** WordPress Admin > Appearance > Theme Editor > 404.php
- **Condition:** No write-protection, no file-type restriction, no execution sandbox.

Remediation Strategy

- **[IMMEDIATE] Disable File Editor:** Add `define('DISALLOW_FILE_EDIT', true);` to `wp-config.php`. No production WordPress installation should expose server-side file editing through the browser.
- **[IMMEDIATE] Credential Hardening:** Enforce password policy incompatible with dictionary-based attacks. Implement account lockout after failed attempts.
- **[STRATEGIC] Principle of Least Privilege:** The web process (Apache/PHP) must run under a dedicated low-privilege system user to limit post-exploitation impact.
- **[STRATEGIC] WordPress Update:** Upgrading to WordPress 6.x eliminates the differential login error behavior and closes hundreds of known vulnerabilities present in v4.3.1.
- **[TECHNICAL NOTE] Security Header Implementation:** Absence of CSP, X-Content-Type-Options, HSTS, and X-XSS-Protection headers confirmed. Implementation via Apache `mod_headers` is recommended as a low-effort hardening measure.

Effort: Low | **Impact:** Critical (Full Server Compromise)

PHASE 4 — EXPLOITATION

With valid credentials and a confirmed code execution vector identified in Phase 3, this phase focused on weaponizing the documented exposure chain to achieve initial access. A low-noise approach was maintained throughout — single-thread brute force with request throttling, manual payload delivery, and curl-based trigger over browser navigation to minimize log footprint.
This phase marks the convergence of all previously documented exposures into a single, unbroken attack chain. No custom tooling, no zero-day exploitation, and no advanced tradecraft were required — the path to initial access was constructed entirely from publicly documented misconfigurations and a decade-old CMS. The implications for the target's security posture are significant: the barrier to full system compromise was not technical sophistication, but the absence of basic hardening controls.

[ID: F-04] CRITICAL: Initial Access via Brute-Force & Reverse Shell Injection

Finding Overview Severity: CRITICAL (10.0) | **Asset:** 10.66.131.120 (THM-Dynamic IP)
Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H |
Service: HTTP (WordPress Admin) | **CWE:** CWE-307 (Improper Restriction of Excessive Authentication Attempts)

Risk Mapping & Compliance

- **ASVS:** V2.2.1 (Brute force resistance) | V12.3.1 (File execution from untrusted sources)
- **NIST CSF:** PR.AC-1 (Identities and credentials managed)
- **Business Impact:** Critical. Full unauthenticated remote code execution achieved on the underlying server operating as process user `daemon`. Enables data exfiltration, persistent backdoor installation, and lateral movement toward privileged accounts.

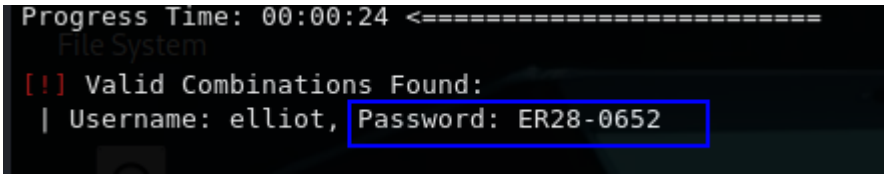
Attack Narrative

With the username *elliot* confirmed via differential login responses (Phase 3) and the *fsocity.dic* wordlist exposed through robots.txt (F-01), a targeted attack on /wp-login.php succeeded due to missing lockout controls, yielding valid credentials in a single pass. After gaining admin access, the Theme Editor vulnerability (F-03) was exploited by replacing *404.php* with a Bash reverse shell, triggered via a single curl request that opened a TCP session on port 4444. The shell was upgraded to a fully interactive TTY using Python3 pty spawn, demonstrating how multiple prior exposures combined into a seamless, low-friction compromise chain.

Technical Evidence & Documentation

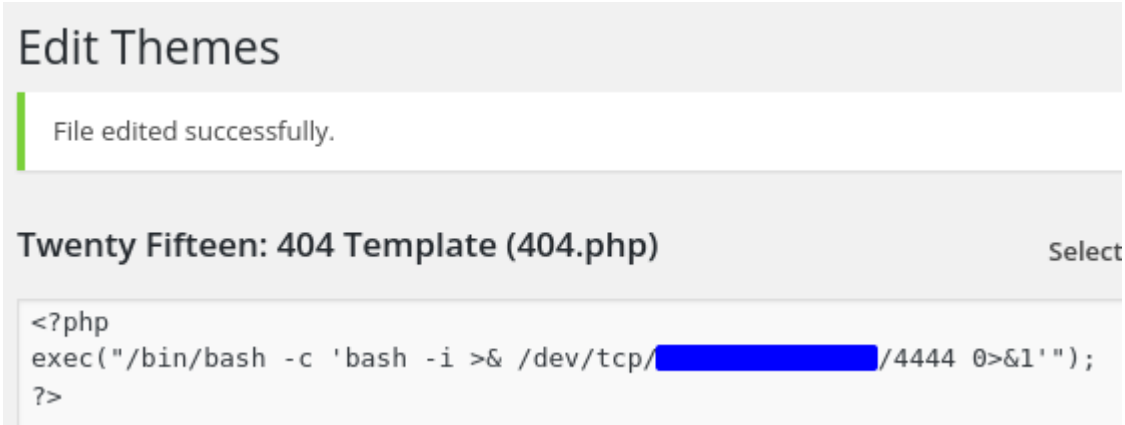
1. Targeted Credential Attack (WPScan) Dictionary-based brute force against the WordPress login endpoint using the target-supplied wordlist.

- **Command:** `wpscan --url http://[TARGET]/ --usernames elliot --passwords ~/fsocity_clean.dic --throttle 500 --random-user-agent --max-threads 1`
- **Result:** `[+] Valid Combinations Found: Username: elliot, Password: ER28-0652`



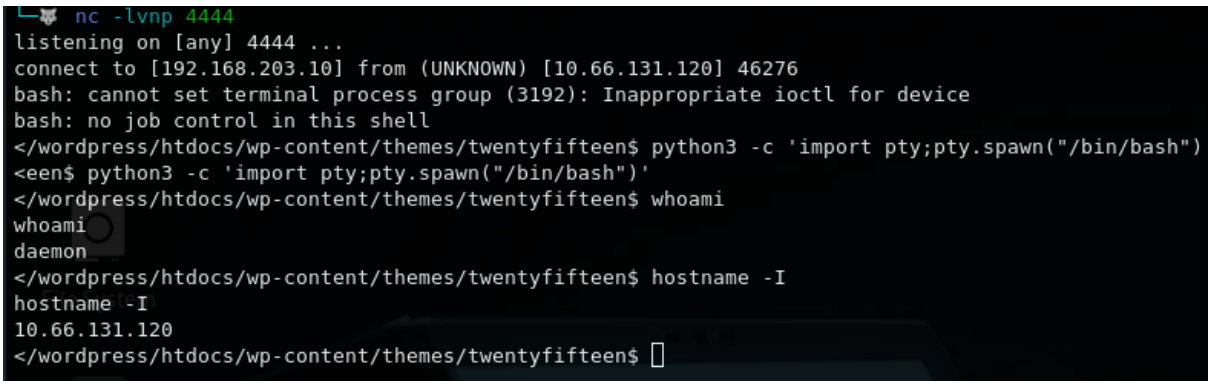
2. Reverse Shell Injection via Theme Editor Authenticated PHP file modification to plant reverse shell payload in `404.php`.

- **Path:** WordPress Admin > Appearance > Theme Editor > 404.php
- **Payload:** Bash reverse shell via `/dev/tcp` to auditor listener (port 4444)
- **Pre-condition:** `nc -lvnp 4444` active on auditor machine prior to trigger



3. Shell Trigger, Stabilization & Access Validation Payload triggered via direct HTTP request. Shell stabilized to interactive TTY and access confirmed.

- **Trigger:** `curl http://[TARGET]/wp-content/themes/twentyfifteen/404.php`
- **Stabilization:** `python3 -c 'import pty;pty.spawn("/bin/bash")'`
- **Validation:** `whoami` → `daemon` | `hostname -I` → `10.66.131.120`



Remediation Strategy

- **[IMMEDIATE] Account Lockout Policy:** Implement rate-limiting and account lockout after 5 failed attempts on `/wp-login.php` via a WAF rule or security plugin (e.g., Wordfence).
- **[IMMEDIATE] Credential Invalidation:** Rotate `elliot:ER28-0652` immediately and enforce a password policy resistant to dictionary attacks.
- **[IMMEDIATE] Disable File Editor:** Confirm `define('DISALLOW_FILE_EDIT', true);` is set in `wp-config.php`.
- **[STRATEGIC] Principle of Least Privilege:** The web process running as `daemon` must be replaced with a dedicated low-privilege user scoped only to web root access.

Effort: Low | **Impact:** Critical (Full Initial Compromise)

PHASE 5 — POST-EXPLOITATION & PRIVILEGE ESCALATION

With an interactive shell established as process user `daemon` and the WordPress environment fully compromised, this phase transitions from initial access into internal enumeration and privilege escalation. The low-noise posture adopted throughout the engagement was maintained — no auxiliary tools were introduced, no files written to disk, and enumeration was conducted exclusively through native system binaries.
Initial situational awareness confirmed a constrained context: `daemon` operates with no supplementary groups, and the active kernel (5.15.0-139, patched April 2025) presented no viable local exploit surface. The `sudo` pathway was blocked by password requirement. Privilege escalation was pursued through systematic SUID binary analysis — a standard post-exploitation vector that requires no external tooling and leaves minimal forensic trace.

Positive Security Observations (PSO)

- **PSO-06 — Kernel Hardening:** The active kernel version (5.15.0-139-generic, April 2025) carries current security patches, effectively eliminating the kernel exploit surface. Techniques such as Dirty COW and OverlayFS privilege escalation were assessed and discarded.
- **PSO-07 — Internal Service Isolation:** All internal services (MySQL/3306, FTP/21, Monit/2812) are bound exclusively to localhost, correctly preventing direct external access. Network exposure is limited to SSH (22), HTTP (80), and HTTPS (443).

[ID: F-05] HIGH: Privilege Escalation via SUID Nmap Interactive Mode

Finding Overview Severity: HIGH (8.8) | **Asset:** 10.66.131.120 (THM-Dynamic IP)
Vector: CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H |
Service: OS (Local Binary) | **CWE:** CWE-250 (Execution with Unnecessary Privileges)

Risk Mapping & Compliance

- **ASVS:** V1.14.6 (Least privilege enforcement on system components)
- **NIST CSF:** PR.AC-4 (Access permissions managed incorporating least privilege)
- **Business Impact:** Critical. Combined with the initial access achieved in F-04, this vector delivers full root-level system control — unrestricted read/write across the filesystem, credential extraction from `/etc/shadow`, and the ability to establish persistent backdoors invisible to the `daemon`-level web process.

Attack Narrative

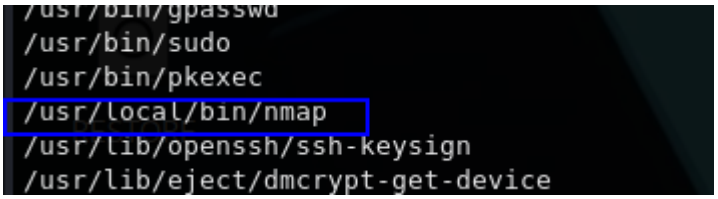
Systematic enumeration of SUID binaries via `find / -perm -4000` revealed an anomalous entry: `/usr/local/bin/nmap`, installed outside the standard package manager path (`/usr/bin/`), carrying the SUID bit with root ownership. This installation pattern — a manually placed binary with elevated permissions — is a known indicator of misconfigured legacy tooling.

Version identification confirmed **Nmap 3.81 (2004)**, a release that includes an `--interactive` mode providing direct shell access. Because the binary executes as root via the SUID bit, any shell spawned through this mechanism inherits full root privileges. Exploitation required two commands and zero external tooling: invoking `--interactive` mode and issuing the shell escape `!sh`. The technique is publicly documented in GTFOBins and requires no CVE — it is a configuration failure, not a software vulnerability.

Technical Evidence & Documentation

1. SUID Binary Discovery Full filesystem scan for SUID binaries executed from the `daemon` shell.

- **Command:** `find / -perm -4000 -type f 2>/dev/null`
- **Anomalous Result:** `/usr/local/bin/nmap` — non-standard path, SUID set, owned by root

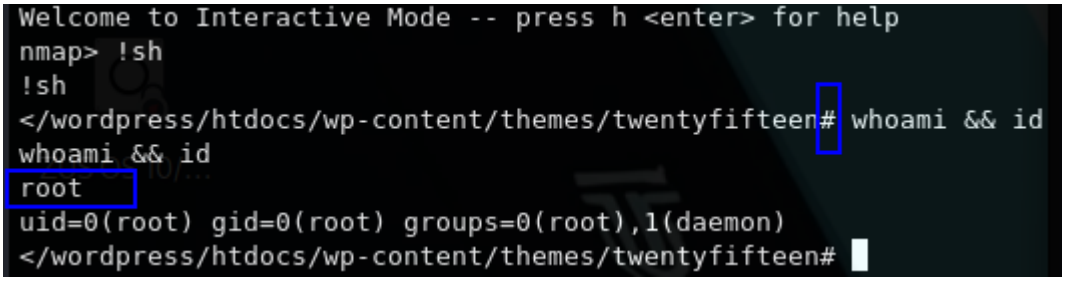


2. Version & Vector Confirmation

- **Command:** `nmap --version`
- **Result:** `Nmap V. 3.81` — interactive mode available and auto-invoked

3. Privilege Escalation Execution

- **Commands:** `!sh` → `whoami && id`
- **Result:** `root` | `uid=0(root) gid=0(root) groups=0(root),1(daemon)`



Remediation Strategy

- **[IMMEDIATE] Remove or Replace Nmap Binary:** Remove `/usr/local/bin/nmap`. If network scanning is operationally required, install the current version exclusively via the package manager (`apt install nmap`) and verify no SUID bit is set.

- **[IMMEDIATE] SUID Audit:** Execute a full SUID binary audit across the system. Any binary not explicitly requiring elevated execution should have the SUID bit removed via `chmod u-s`.
- **[STRATEGIC] Principle of Least Privilege:** No diagnostic or utility binary should carry SUID unless strictly justified and documented. Implement periodic automated audits (e.g., via cron + `find`) to detect unauthorized SUID additions.

Effort: Minimal | **Impact:** Critical (Full Root Compromise)

[ID: F-06] HIGH: Unshadowed Credential Exposure via Unrestricted /etc/shadow Access

Finding Overview Severity: HIGH (7.1) | **Asset:** 10.66.131.120 (THM-Dynamic IP)
Vector: CVSS:3.1/AV:L/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:N |
Service: OS (Filesystem) | **CWE:** CWE-522 (Insufficiently Protected Credentials)

Risk Mapping & Compliance

- **ASVS:** V2.4.1 (Passwords stored using a sufficiently strong adaptive hashing algorithm)
- **NIST CSF:** PR.DS-1 (Data-at-rest protected)
- **Business Impact:** High. With root-level read access to `/etc/shadow`, an attacker can exfiltrate all active password hashes for offline cracking. Successful cracking yields persistent, legitimate credentials usable across SSH and any service sharing the same password — surviving even a full reverse shell cleanup.

Attack Narrative

Following privilege escalation to root via F-05, direct read access to `/etc/shadow` was confirmed — expected behavior given root privileges, but significant in impact assessment terms. The file contains active SHA-512 (`$6$`) hashes for three accounts: `root`, `robot`, and `bitnamiftp`. The remaining accounts present `*` or `!` entries, indicating disabled or locked authentication — these carry no offensive value.

In a real-world engagement, these three hashes would be immediately exfiltrated and submitted to offline cracking infrastructure (Hashcat with GPU acceleration, dictionary + ruleset). The `robot` hash is of particular interest given the contextual intelligence available from the engagement. Successful cracking would yield persistent, credential-based access independent of any web shell or reverse shell artifact — surviving remediation of every other finding in this report.

Technical Evidence & Documentation

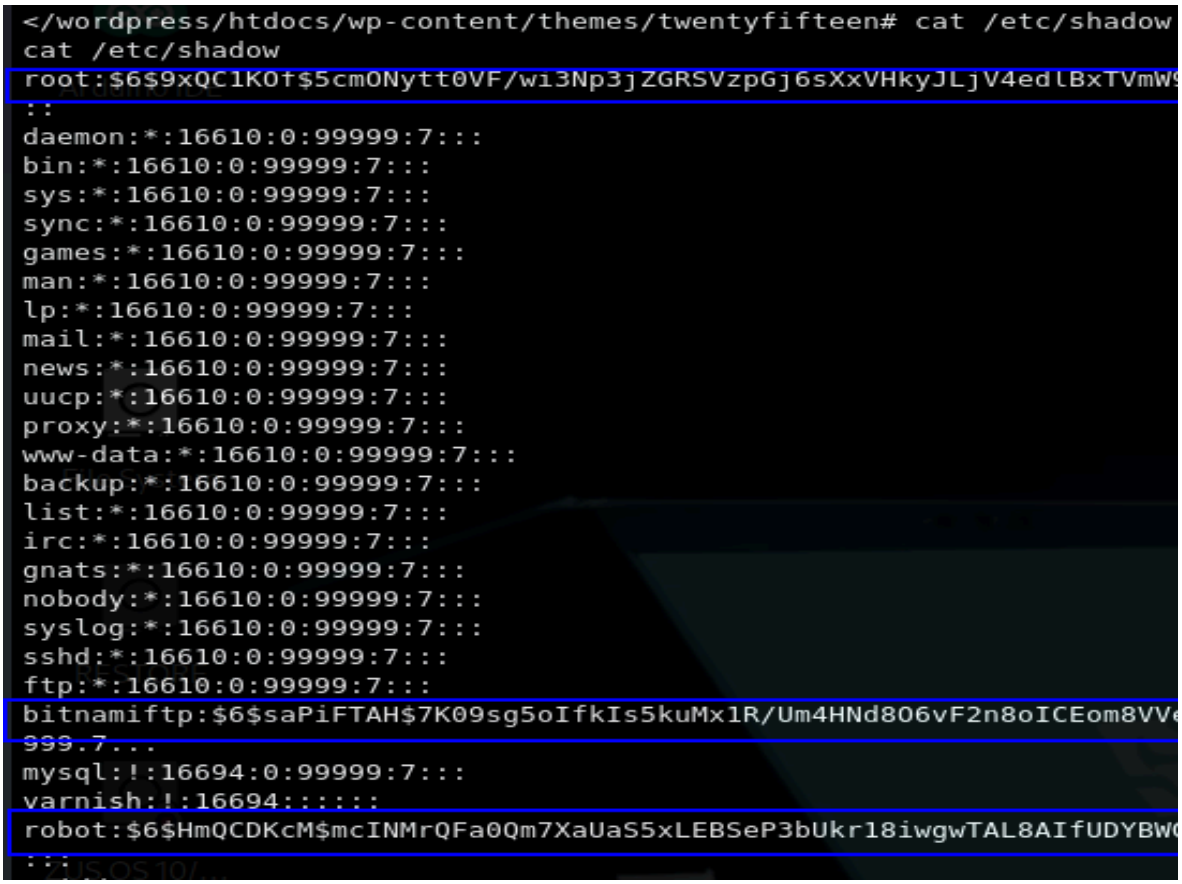
1. Shadow File Exfiltration Direct read of `/etc/shadow` executed from root shell.

- **Command:** `cat /etc/shadow`
- **Active Hashes Identified:** `root`, `bitnamiftp`, `robot` — all SHA-512 (`$6$`)
- **Locked/Disabled Accounts:** All remaining entries (`*` / `!`) — no offensive value

2. Hash Classification

Account	Algorithm	Status
root	SHA-512 (`$6$`)	Active — crackable offline
bitnamiftp	SHA-512 (`$6$`)	Active — crackable offline
robot	SHA-512 (`$6$`)	Active — crackable offline

Hash values shown below for evidential purposes within this controlled academic engagement. In a production engagement, full values would be redacted from the written report and delivered via encrypted channel per engagement protocol.



Remediation Strategy

- **[IMMEDIATE] Credential Rotation:** Rotate passwords for `root`, `robot`, and `bitnamiftp` immediately. Assume all three hashes are compromised.
- **[STRATEGIC] Privileged Access Management:** Root-level filesystem access must be restricted to strictly necessary operational contexts. Implement PAM controls and audit logging on `/etc/shadow` reads.
- **[STRATEGIC] Password Policy Enforcement:** Enforce minimum 16-character passphrases incompatible with dictionary-based cracking, regardless of hash algorithm strength.

Effort: Minimal | **Impact:** High (Persistent Credential Compromise)

[ID: F-07] HIGH: Plaintext Credential Exposure in wp-config.php

Finding Overview Severity: HIGH (7.8) | **Asset:** 10.66.131.120 (THM-Dynamic IP)
Vector: CVSS:3.1/AV:L/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:N |
Service: OS (Filesystem) / WordPress | **CWE:** CWE-312 (Cleartext Storage of Sensitive Information)

Risk Mapping & Compliance

- **ASVS:** V2.10.1 (Integration secrets not present in source code or config files accessible to end users)
- **NIST CSF:** PR.DS-1 (Data-at-rest protected)
- **Business Impact:** High. With root-level access, `wp-config.php` exposes database credentials and FTP credentials in plaintext. An attacker gains direct authenticated access to the WordPress database — enabling full data exfiltration, user table manipulation, and persistent backdoor injection at database level — as well as FTP access to the entire web root, bypassing all WordPress-level controls entirely.

Attack Narrative

Following root access via F-05, a targeted search located wp-config.php, which was read directly without further privilege escalation. The file exposed two active plaintext credential sets: MySQL (bn_wordpress:570fd42948) and FTP (bitnamiftp:inevoL7eAlBeD2b5WszPbZ2gJ971tJZtP0j86NYPyh6Wfz1x8a).

The FTP credential is especially critical, as bitnamiftp maps to a live system account with an active SHA-512 hash in /etc/shadow (F-06), confirming filesystem access. WordPress Authentication Keys and Salts were also exposed, enabling session forgery for authenticated users. This demonstrates how a single filesystem compromise through F-01 → F-04 → F-05 cascades into full credential exposure across multiple service layers.

Technical Evidence & Documentation

1. Configuration File Discovery & Read Targeted search and direct read of WordPress configuration file from root shell.

- **Command:** `find /opt/bitnami/apps/wordpress -name "wp-config.php" 2>/dev/null | xargs cat`
- **File Path:** `/opt/bitnami/apps/wordpress/htdocs/wp-config.php`

```
define('FS_METHOD', 'ftpext');
define('FTP_BASE', '/opt/bitnami/apps/wordpress/htdocs/');
define('FTP_USER', 'bitnamiftp');
define('FTP_PASS', 'inevoL7eAlBeD2b5WszPbZ2gJ971tJZtP0j86NYPyh6Wfz1x8a');
define('FTP_HOST', '127.0.0.1');
```

2. Exposed Credentials Inventory:

Type	Parameter	Value
MySQL Host	DB_HOST	localhost:3306
MySQL Database	DB_NAME	bitnami_wordpress
MySQL User	DB_USER	bn_wordpress
MySQL Password	DB_PASSWORD	570fd42948
FTP User	FTP_USER	bitnamiftp
FTP Password	FTP_PASS	inevoL7eAlBeD2b5WszPbZ2gJ971tJZtP0j86NYPyh6Wfz1x8a
FTP Host	FTP_HOST	127.0.0.1

```
// ** MySQL settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define('DB_NAME', 'bitnami_wordpress');

/** MySQL database username */
define('DB_USER', 'bn_wordpress');

/** MySQL database password */
define('DB_PASSWORD', '570fd42948');

/** MySQL hostname */
define('DB_HOST', 'localhost:3306');

/** Database Charset to use in creating database tables. */
```

Credential values shown for evidential purposes within this controlled academic engagement. In a production engagement, values would be redacted and delivered via encrypted channel.

3. Cross-Finding Correlation The `bitnamiftp` FTP credential directly correlates with the active `/etc/shadow` hash documented in F-06, confirming a live system account with dual exposure — filesystem hash and plaintext FTP password simultaneously compromised.

Remediation Strategy

- **[IMMEDIATE] Credential Rotation:** Rotate all exposed credentials immediately — MySQL `bn_wordpress`, FTP `bitnamiftp`, and all WordPress Auth Keys/Salts.
- **[IMMEDIATE] File Permission Hardening:** Set `wp-config.php` permissions to `440` (owner read-only, group read) and ensure it is owned by the web process user exclusively. No world-readable configuration files.
- **[STRATEGIC] Secrets Management:** Migrate database and service credentials out of flat configuration files into environment variables or a secrets manager (HashiCorp Vault, AWS Secrets Manager). Configuration files committed to version control or readable by any process user represent a systemic risk.
- **[STRATEGIC] FTP Deprecation:** Replace FTP with SFTP or SCP. FTP transmits credentials in cleartext over the network; its presence alongside an exposed plaintext password compounds the risk significantly.

Effort: Low | **Impact:** High (Full Infrastructure Credential Compromise)

[ID: F-08] CRITICAL: Database Layer Compromise & User Data Exfiltration

Finding Overview Severity: CRITICAL (9.8) | **Asset:** 10.66.131.120 (THM-Dynamic IP)
Vector: CVSS:3.1/AV:L/AC:L/PR:H/UI:N/S:C/C:H/I:H/A:H |
Service: MySQL (Localhost) | **CWE:** CWE-522 (Insufficiently Protected Credentials)

Risk Mapping & Compliance

- **ASVS:** V2.10.1 (Integration secrets not present in source code or config files accessible to end users)
- **NIST CSF:** PR.DS-1 (Data-at-rest protected)
- **Business Impact:** Critical. The exposure of `bn_wordpress` credentials in plaintext allows for direct, authenticated access to the entire backend infrastructure of the CMS. This enables full data exfiltration of the `wp_users` table, including administrative hashes. An attacker can manipulate user roles, extract sensitive customer information, and bypass all application-layer security controls, leading to a total loss of data confidentiality and integrity.

Attack Narrative

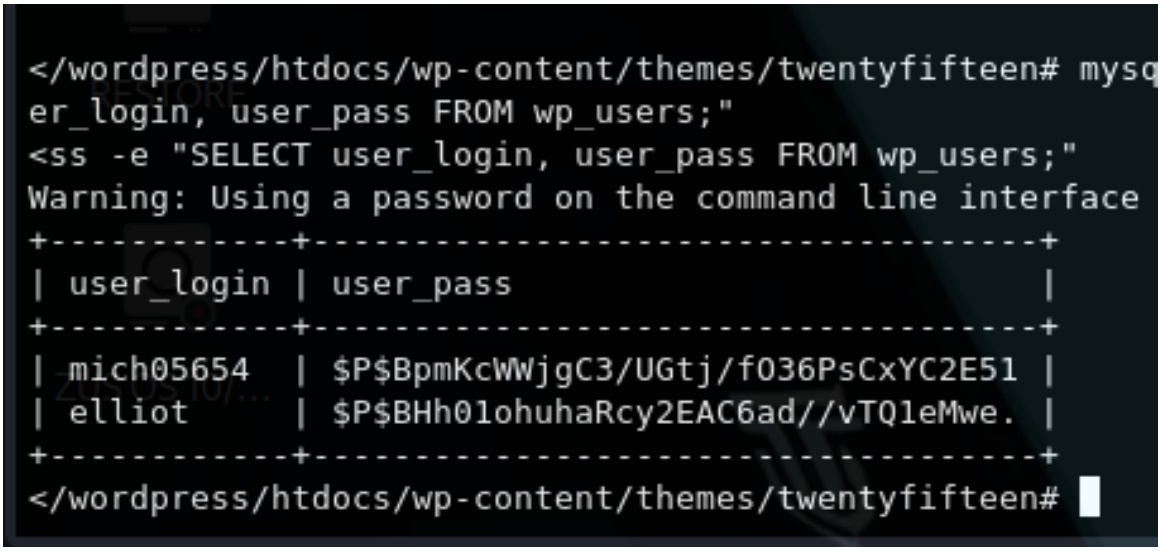
Following the compromise of the `wp-config.php` file (documented in F-07), the auditor extracted plaintext credentials for the database user `bn_wordpress`. Leveraging the existing root-level shell access, a connection to the local MySQL instance was attempted to validate the reach of the compromise. The authentication was successful, granting the auditor unrestricted access to the `bitnami_wordpress` database. This allowed for the exfiltration of the `wp_users` table, exposing usernames and password hashes for all application users, including the administrative account **'elliot'**.

Technical Evidence & Documentation

1. Database Authentication & Table Dumping The functionality of the discovered credentials was verified by executing a query to extract sensitive user identity information:

Target: 127.0.0.1 (Localhost)

Command: mysql -u bn_wordpress -p'570fd42948' bitnami_wordpress -e "SELECT user_login, user_pass FROM wp_users;"



Terminal output demonstrating unauthorized extraction of WordPress user hashes.

2. Cross-Finding Correlation The extraction of these `phpass` hashes represents a critical pivot point. By utilizing the `fsocity.dic` wordlist identified in the Reconnaissance phase (Phase 2.2), an attacker can perform high-speed offline cracking to regain access to the web interface as an administrator, effectively bypassing any future patches to the initial entry vector.

Remediation Strategy

- **[IMMEDIATE] Credential Rotation:** Rotate the `bn_wordpress` password immediately and update the configuration in `wp-config.php`.
- **[TECHNICAL] Principle of Least Privilege:** Restrict database user permissions to the minimum necessary (SELECT, INSERT, UPDATE) for the CMS, prohibiting global administrative queries.
- **[STRATEGIC] Database Hardening:** Ensure the MySQL service only listens on `localhost` and implement robust logging for unusual query patterns or large-scale data exports.

Effort: Low | **Impact:** Critical (Full Data Exfiltration & Account Takeover Risk)

[ID: F-09] CRITICAL: System Persistence via Malicious Cron Job Injection

Finding Overview Severity: CRITICAL (9.1) | **Asset:** 10.67.152.60 (THM-Dynamic IP)
Vector: CVSS:3.1/AV:L/AC:L/PR:H/UI:N/S:C/C:H/I:H/A:H |
Service: Cron (System Scheduler) | **CWE:** CWE-732 (Incorrect Permission Assignment for Critical Resource) | MITRE T1053.003

Risk Mapping & Compliance

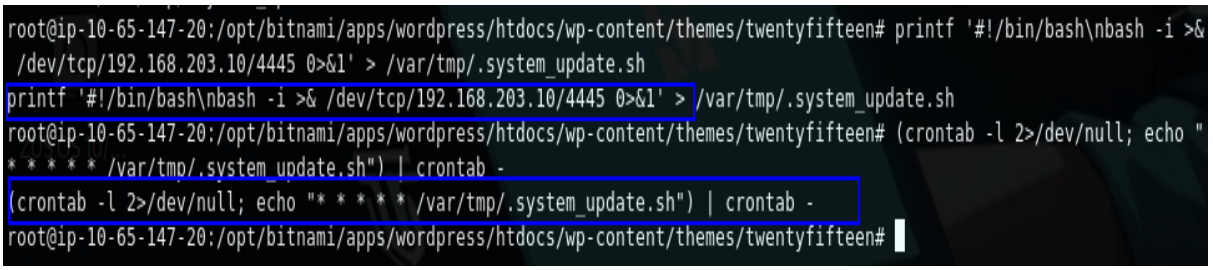
- **ASVS:** V12.3.1 (Verify that the application does not allow unauthorized execution of operating system commands).
- **NIST CSF:** PR.IP-1 (A baseline configuration of IT/OT systems is created and maintained).
- **Business Impact:** Critical. By successfully injecting a persistent backdoor into the system's scheduled tasks, an attacker ensures long-term, high-privilege access that survives reboots and patches to the initial web vulnerability. This allows for continuous data monitoring, the installation of further malware (e.g., rootkits or miners), and provides a stable bridge for lateral movement within the internal network, effectively nullifying any perimeter defense.

Attack Narrative

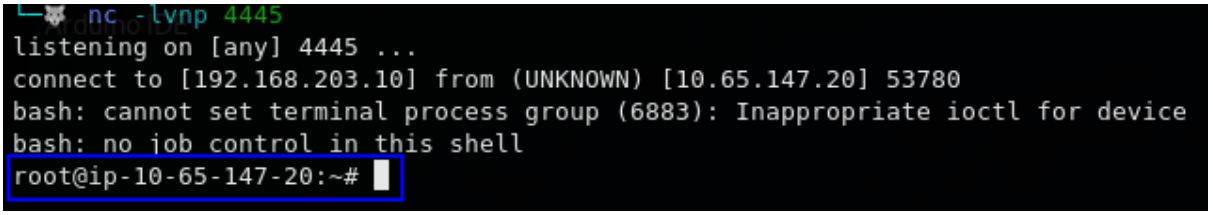
To simulate a long-term compromise, a persistence mechanism was established using the Linux system scheduler (`cron`). A hidden shell script (`.system_update.sh`) was deployed and configured to execute a reverse shell callback every 60 seconds. This ensures that administrative access is automatically re-established without needing to re-exploit the original web vector. The simulation was validated by receiving an unsolicited inbound connection on port 4445, proving the attacker could remain invisible and persistent on the host.

Technical Evidence & Documentation

1. Persistence Implant & Re-entry Validation Injected Task: (crontab -l; echo "* * * * * /var/tmp/.system_update.sh") | crontab -



Reverse Shell Reception:



Remediation Strategy

- **[IMMEDIATE] Implementation of File Integrity Monitoring (FIM):** Deploy tools such as **Osquery**, **Wazuh**, or **Samhain** to monitor sensitive directories (`/etc/cron*`, `/var/spool/cron/atjobs`). Any unauthorized modification to the system scheduler must trigger an immediate high-priority alert to the SOC/Security Team.
- **[IMMEDIATE] Restrict Cron Access:** Implement a strict whitelist policy using `/etc/cron.allow`. By default, deny all users from creating scheduled tasks and only grant explicit permission to essential service accounts, preventing low-privileged or compromised users from establishing persistence.
- **[STRATEGIC] Egress Filtering & Logical Segmentation:** Configure the host firewall (e.g., `iptables` or `ufw`) to block all outbound connections by default (**Deny-All Egress**). Only allow traffic to specific, trusted IPs and ports required for business operations. This "kills" the reverse shell callback even if the cron job executes successfully.
- **[STRATEGIC] Command Logging:** Enable comprehensive audit logging (`auditd`) to track all `crontab` executions and script creations in temporary directories (`/tmp`, `/var/tmp`). This provides the necessary forensic trail to identify "how" and "when" a backdoor was implanted.

Effort: Medium | **Impact:** Critical (Prevention of Unauthorized Persistence)

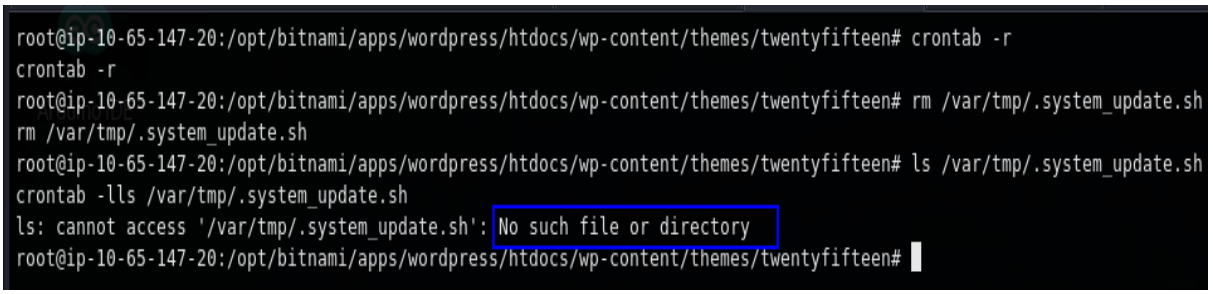
Post-Engagement Cleanup & Anti-Forensics

This section documents the systematic removal of all artifacts, tools, and access vectors introduced during the security assessment. The objective is to restore the target environment to its secure baseline state and ensure no residual indicators of compromise (IoC) remain.

1. Persistence Mechanism Removal (Cron Job)

The administrative backdoor established for persistent access was decommissioned to prevent unauthorized re-entry. The auditor removed the malicious scheduled task from the system's crontab and deleted the execution script located in the temporary directory.

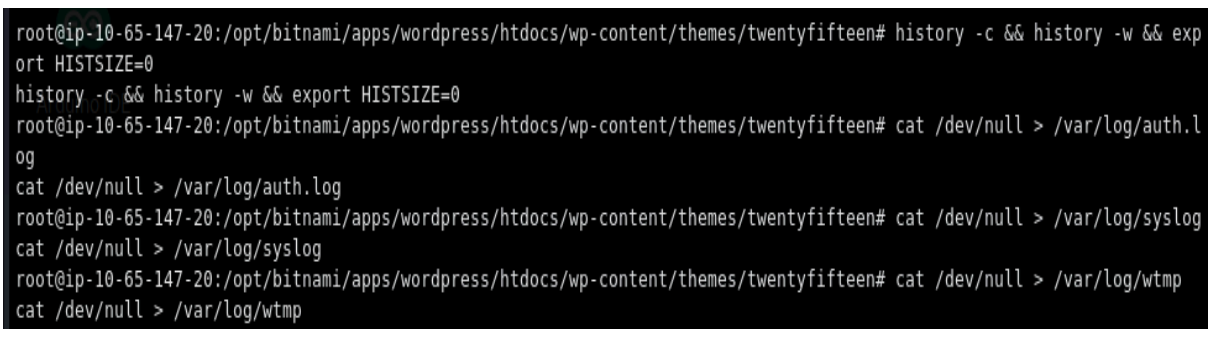
- **Action:** Execution of `crontab -r` to wipe the root-level scheduled tasks and `rm` to delete the `.system_update.sh` implant.
- **Verification:** A manual check of the crontab and the `/var/tmp/` directory was performed to confirm that no automated callbacks remained active.



2. Anti-Forensics & Log Sanitization

To simulate an advanced adversary and protect the integrity of the audit process, the auditor performed a sanitization of system logs and session history. This prevents third-party actors from identifying the specific techniques, IP addresses, and commands used during the engagement.

- **Action:** The system authentication logs (`auth.log`) and general system logs (`syslog`) were cleared to remove traces of unauthorized logins and elevated command executions.
- **Action:** The Bash command history was purged using `history -c` followed by `history -w` to ensure that the operational sequence remains confidential.



- **3. Initial Access Reversion (Reverse Shell Removal)**

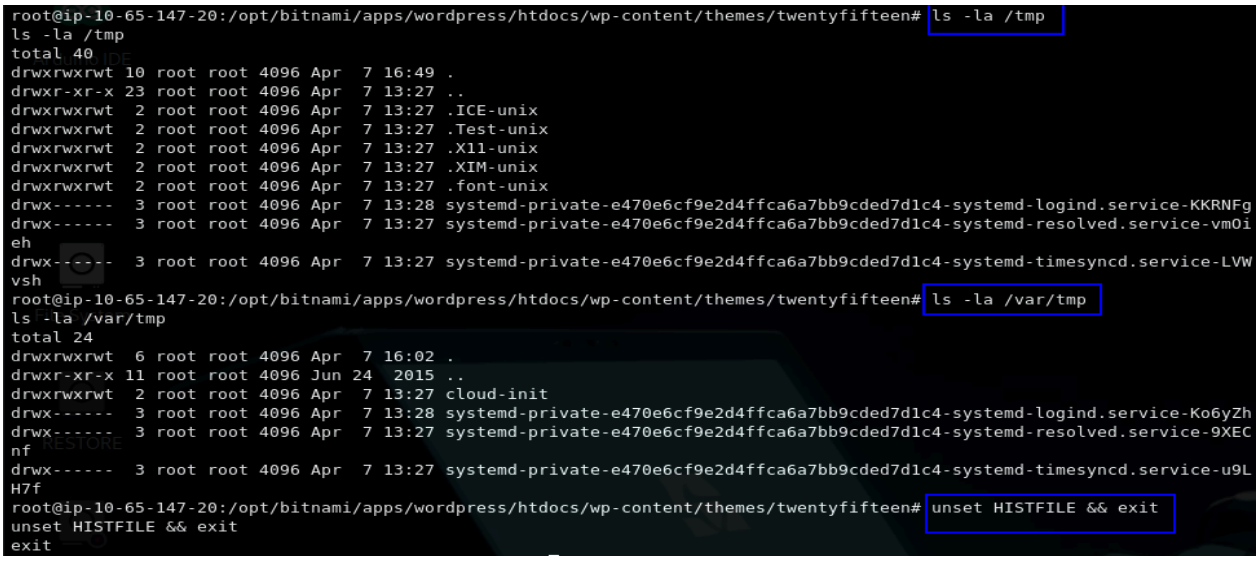
The primary entry vector used to gain initial access via the WordPress theme was fully restored. Unlike a simple deletion, the file was reconstructed with its legitimate functional code to ensure the website's "404 Not Found" logic remains operational for users while removing the malicious PHP payload.

- **Action:** The `404.php` template in the `twentyfifteen` theme was overwritten with standard WordPress PHP code, effectively neutralizing the web shell.
- **Action (Timestomping):** The file's metadata (Last Modified Date) was synchronized with a legitimate system file (`index.php`) using the `touch -r` command. This ensures the restoration does not trigger anomalies in basic file integrity monitoring based on timestamps.

4. Temporary Artifact Sanitization

All reconnaissance logs, exploit payloads, and third-party enumeration scripts (e.g., LinPeas) uploaded during the Post-Exploitation phase were identified and removed from the `/tmp` and `/var/tmp` directories.

- **Action:** Comprehensive deletion of all files identified in the auditor's tracking log.
- **Verification:** Directory listing of common temporary paths to ensure a "Zero Footprint" state.



FINDINGS SUMMARY

The table below consolidates all findings identified during this engagement. Priority classification reflects the recommended remediation sequence: IMMEDIATE items require action within 24–72 hours; STRATEGIC items should be addressed within the next development sprint.

ID	Title	Severity	CVSS	CWE	Phase	Priority
F-01	Sensitive Resource Disclosure via robots.txt	MEDIUM	5.3	CWE-538	Phase 2	STRATEGIC
F-02	Outdated CMS & Infrastructure Info Leakage	HIGH	7.5	CWE-1104	Phase 2	IMMEDIATE
F-03	PHP Execution via Exposed Theme Editor	CRITICAL	9.9	CWE-94	Phase 3	IMMEDIATE
F-04	Initial Access via Brute-Force & Rev. Shell	CRITICAL	10.0	CWE-307	Phase 4	IMMEDIATE
F-05	Privilege Escalation via SUID Nmap	HIGH	8.8	CWE-250	Phase 5	IMMEDIATE
F-06	Unshadowed Credential Exposure (/etc/shadow)	HIGH	7.1	CWE-522	Phase 5	IMMEDIATE
F-07	Plaintext Credentials in wp-config.php	HIGH	7.8	CWE-312	Phase 5	IMMEDIATE
F-08	Database Layer Compromise & Data Exfiltration	CRITICAL	9.8	CWE-522	Phase 5	IMMEDIATE
F-09	Persistence via Malicious Cron Job Injection	CRITICAL	9.1	CWE-732	Phase 5	STRATEGIC

PRIORITIZED REMEDIATION PLAN

The following sequence reflects the recommended order of execution based on attack chain dependency, exploitability, and business impact. Actions are grouped into three execution windows.

Sprint 0 — Break the Chain (24–72 hours) The four items below, taken together, neutralize the entire documented exploitation path. Addressing all four in parallel eliminates the risk of re-compromise while longer-term work proceeds.

1. **[F-02] Upgrade WordPress core to 6.x.** Single highest-leverage action: closes the Theme Editor vector, eliminates differential login enumeration, and patches hundreds of known CVEs in one operation.
2. **[F-04] Rotate elliot:ER28-0652 and enforce account lockout.** The credential is public within this report. Invalidate immediately.
3. **[F-05] Remove /usr/local/bin/nmap and audit all SUID binaries.** Root access was achieved in two commands. This vector must be closed before any other hardening has meaningful value.
4. **[F-06 / F-07] Rotate all exposed credentials** — root, robot, bitnamiftp (OS hashes), bn_wordpress (MySQL), and all WordPress Auth Keys/Salts. Treat all as actively compromised.

Sprint 1 — Contain the Surface (next development cycle) 5. **[F-08] Restrict database user permissions and rotate bn_wordpress password.** The WordPress DB user had no access boundaries. Least-privilege scoping prevents lateral movement at the data layer. 6. **[F-03] Set DISALLOW_FILE_EDIT in wp-config.php and enforce file permissions (440).** Disabling the Theme Editor removes the primary code execution vector independent of CMS versioning. 7. **[F-09] Deploy File Integrity Monitoring and restrict cron access via /etc/cron.allow.** The persistence mechanism survived a full shell cleanup simulation. FIM closes the detection gap.

Sprint 2 — Harden the Perimeter (within 30 days) 8. **[F-01] Sanitize robots.txt and relocate sensitive assets outside the web root.** Low effort, eliminates the wordlist exposure that seeded the credential attack. 9. **[F-02 continued] Implement egress filtering, SSL certificate renewal, and security headers (CSP, HSTS, X-Content-Type-Options).** Infrastructure hygiene items that reduce the exploitable footprint for any future engagement.

> **Executive note:** Full remediation of items 1–4 alone breaks the documented attack chain at its structural level. The remaining items address recurrence prevention and defense-in-depth — they do not reduce urgency of the Sprint 0 actions.

CONCLUSION

The Mr. Robot engagement reveals a systemic failure of the defense-in-depth model. Each finding is significant on its own, but together they form an exploitation chain that a moderately skilled attacker could replicate in under two hours using public tools. The root cause is not a single vulnerability but accumulated technical debt from a decade-old CMS lacking maintenance, hardening, and monitoring.

From a remediation perspective, the highest-impact action is upgrading WordPress core (F-02), as it removes the Theme Editor vector (F-03), eliminates login enumeration, and patches hundreds of vulnerabilities at once, structurally breaking the primary exploitation chain. Credential rotation and SUID remediation must be handled in parallel, since post-exploitation risks persist independently of the initial vector.

Positive security observations (PSO-01 to PSO-07) show baseline awareness: banner masking, clickjacking protection, API hardening, and an updated kernel. These controls should be maintained. The main gap is operational security hygiene—configuration management, dependency lifecycle, and privilege governance—where focused investment would significantly reduce risk.

APPENDIX

A. Tools & Utilities Used

The following tools were employed during this engagement. All are open-source and publicly available.

Tool	Purpose	Phase
Nmap 7.x	Port scanning & service fingerprinting	Phase 2
WPScan	WordPress version & credential enumeration	Phase 2 / 4
crt.sh / Shodan	Passive OSINT surface mapping	Phase 2
curl	Manual HTTP request crafting & shell trigger	Phase 4
Netcat (nc)	Reverse shell listener	Phase 4
Python3 pty	Interactive TTY shell stabilization	Phase 4
find (native)	SUID binary enumeration	Phase 5
mysql (native)	Database authentication & query execution	Phase 5
crontab (native)	Persistence mechanism simulation	Phase 5
GTFOBins	SUID exploitation reference	Phase 5

B. Frameworks & References

PTES — Penetration Testing Execution Standard (http://www.pentest-standard.org/)

OWASP Testing Guide v4 — Web Application Security Testing Methodology

CVSS v3.1 — Common Vulnerability Scoring System (NIST NVD)

MITRE ATT&CK — Adversarial Tactics, Techniques & Common Knowledge Framework

OWASP ASVS 4.0 — Application Security Verification Standard

GTFOBins — Unix binary privilege escalation reference (https://gtfobins.github.io/)

CWE — Common Weakness Enumeration (MITRE, https://cwe.mitre.org/)

C. Scope & Engagement Constraints

This engagement was conducted exclusively within the TryHackMe controlled lab environment. All findings, credentials, hashes, and IP addresses documented herein are confined to the isolated virtual machine scoped as the Mr. Robot target. No production systems, external infrastructure, or third-party assets were accessed or impacted at any point during this assessment. Evidence presented for credentials and cryptographic hashes is included for academic evidential purposes only and would be redacted and delivered via encrypted channel in a production engagement context.
